

1. ¿Cuál es la diferencia entre una lista y una tupla en Python?

La diferencia principal es la mutabilidad:

Listas: Son mutables. Esto significa que puedes modificarlas después de crearlas (añadir, eliminar o cambiar elementos). Se usan cuando necesitas una colección de datos que va a cambiar.

Tuplas: Son inmutables. Una vez creadas, no se pueden modificar, añadir ni eliminar elementos. Se usan para datos que deben permanecer constantes (como coordenadas o días de la semana).

Además, las tuplas consumen ligeramente menos memoria y son un poco más rápidas que las listas. La sintaxis de las tuplas es `()` (paréntesis) en cambio las listas son `[]` (corchetes).

2. ¿Cuál es el orden de las operaciones?

Python sigue el orden matemático estándar, conocido comúnmente por el acrónimo PEMDAS (Paréntesis, Exponentes, Multiplicación/División, Suma/Resta).

1- Paréntesis → `()`

Ej. $(2+3) * 4 = 20$

2- Exponente → `**`

Ej. $2 ** 3 = 8$

3- Multiplicación / División → `*` / `//` / `%`

Ej. $10 / 2 = 5$

4- Suma / Resta → `+` / `-`

Ej. $5 + 3 = 8$

5- Comparaciones → `==` / `!=` / `>` / `<`

Ej. $5 > 3 \rightarrow \text{True}$

6- Negación lógica → `not`

Ej. `Not True` → `False`

7- Y lógico → `and`

Ej. `True and False` → `False`

8- O lógico → or

Ej. True or False → True

3. ¿Qué es un diccionario Python?

Un diccionario es una estructura de datos que almacena información en pares de clave-valor (key-value).

- Se definen usando llaves {}.
- Son mutables (puedes añadir o cambiar pares clave-valor).
- Desde Python 3.7, los diccionarios mantienen el orden en el que se insertaron los elementos.
- Las claves deben ser inmutables y únicas (como cadenas de texto, números o tuplas), mientras que los valores pueden ser de cualquier tipo de dato (incluso otras listas o diccionarios).

Ej. usuario = {"nombre": "Ana", "edad": 28, "es_estudiante": True}

4. ¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

Ambos son utilizados para ordenar datos, pero funcionan de manera muy diferente.

lista.sort() (Método):

- Solo funciona con listas.
- Ordena la lista en el mismo lugar (in-place), lo que significa que modifica la lista original directamente.
- Devuelve None (no devuelve una nueva lista).

sorted(iterable) (Función):

- Funciona con cualquier iterable (listas, tuplas, cadenas de texto, diccionarios, etc.).
- Devuelve una nueva lista con los elementos ordenados.
- Deja el iterable original sin cambios.

Ej.

```
numeros = [3, 1, 2]
```

```
numeros.sort()    # 'numeros' ahora es [1, 2, 3]. La original se perdió.
```

```
tupla_numeros = (3, 1, 2)
```

```
nueva_lista = sorted(tupla_numeros) # Devuelve [1, 2, 3]. 'tupla_numeros' sigue siendo (3, 1, 2).
```

5. ¿Qué es un operador de reasignación?

Un operador de reasignación (que también puede ser llamado operador de asignación aumentada) combina una operación matemática con la asignación de un valor a una variable. Es una forma abreviada y mas limpia de actualizar el valor de una variable basándose en su valor actual.

En lugar de escribir: $x = x + 5$

Puedes usar el operador de reasignación: $x += 5$

Ejemplos comunes:

$+=$ (Suma y asigna): $x += 3$ es lo mismo que $x = x + 3$

$-=$ (Resta y asigna): $x -= 2$ es lo mismo que $x = x - 2$

$*=$ (Multiplica y asigna): $x *= 4$ es lo mismo que $x = x * 4$

$/=$ (Divide y asigna): $x /= 2$ es lo mismo que $x = x / 2$